Python Learning Notes

A concise reference covering core Python concepts — syntax, data structures, functions, OOP, and best practices.

Topics	Basics → OOP → Best Practices
Pages	10
Level	Beginner → Intermediate

1. Python Basics & Variables

Python is a dynamically typed, interpreted language. Variables don't need explicit type declarations — the type is inferred at assignment and can change at runtime.

Variables and Types

```
x = 10          # int
y = 3.14         # float
name = "Alice"   # str
is_valid = True  # bool
nothing = None   # NoneType

print(type(x))   # <class 'int'>
```

Basic Operators

- **Arithmetic:** + - * / // % ** (// is floor division, ** is power)
- **Comparison:** == != > < >= <=
- **Logical:** and, or, not
- **Assignment:** = += -= *= /= //= %= **=

String Formatting (f-strings)

```
name = "Bob"
age = 25
print(f"{name} is {age} years old")
print(f"{3.14159:.2f}")  # 3.14 (2 decimal places)
```

Tip: f-strings (Python 3.6+) are the preferred way to format strings — they're faster and more readable than .format() or % formatting.

2. Core Data Structures

List — ordered, mutable

```
fruits = ["apple", "banana", "cherry"]
fruits.append("date")        # add to end
fruits.insert(0, "avocado")  # insert at index
fruits.remove("banana")      # remove by value
fruits[1:3]                  # slicing
fruits.sort()                # in-place sort
```

Tuple — ordered, immutable

```
point = (3, 4)
x, y = point   # unpacking
# point[0] = 5  -> TypeError, tuples are immutable
```

Dictionary — key/value pairs

```
person = {"name": "Ann", "age": 30}
person["email"] = "ann@example.com"   # add key
person.get("age", 0)                  # safe access with default
for k, v in person.items():
    print(k, v)
```

Set — unordered, unique elements

```
a = {1, 2, 3}
b = {2, 3, 4}
a | b   # union -> {1, 2, 3, 4}
a & b   # intersection -> {2, 3}
a - b   # difference -> {1}
```

Structure	Ordered	Mutable	Duplicates
list	Yes	Yes	Yes
tuple	Yes	No	Yes
dict	Yes (3.7+)	Yes	Unique keys
set	No	Yes	No

3. Control Flow

Conditionals

```
age = 20
if age < 13:
    category = "child"
elif age < 20:
    category = "teen"
else:
    category = "adult"
```

Loops

```
for i in range(5):
    print(i)          # 0 1 2 3 4

for fruit in ["apple", "kiwi"]:
    print(fruit)

n = 0
while n < 3:
    print(n)
    n += 1
```

Loop Control

- **break** — exits the loop immediately
- **continue** — skips to the next iteration
- **else** on a loop — runs only if the loop completes without a break

Comprehensions

```
squares = [x**2 for x in range(10)]
evens = [x for x in range(20) if x % 2 == 0]
square_map = {x: x**2 for x in range(5)}
unique_lengths = {len(w) for w in ["hi", "bye", "ok"]}
```

Comprehensions are more Pythonic and often faster than building a list with a for-loop and .append() calls.

4. Functions

Defining Functions

```
def greet(name, greeting="Hello"):
    """Return a greeting string."""
    return f"{greeting}, {name}!"

greet("Sam")                # "Hello, Sam!"
greet("Sam", greeting="Hi") # "Hi, Sam!"
```

*args and **kwargs

```
def total(*args, **kwargs):
    print(args)     # tuple of positional args
    print(kwargs)   # dict of keyword args

total(1, 2, 3, tax=0.1, discount=5)
```

Lambda Functions

```
square = lambda x: x ** 2
sorted(names, key=lambda n: len(n))
```

Scope: LEGB Rule

- **Local** — names defined inside the current function
- **Enclosing** — names in an outer (enclosing) function
- **Global** — names defined at the module level
- **Built-in** — names Python provides by default (len, print, etc.)

Tip: avoid mutable default arguments like `def f(x, items=[])` — the same list is reused across calls, causing subtle bugs. Use `items=None` and create the list inside the function instead.

5. Object-Oriented Programming — Classes

Defining a Class

```
class Dog:
    species = "Canis familiaris"   # class attribute

    def __init__(self, name, age):
        self.name = name           # instance attribute
        self.age = age

    def bark(self):
        return f"{self.name} says woof!"

rex = Dog("Rex", 3)
print(rex.bark())
```

Inheritance

```
class Puppy(Dog):
    def __init__(self, name, age):
        super().__init__(name, age)
        self.is_puppy = True

    def bark(self):                 # method override
        return f"{self.name} yips!"
```

Key Dunder (Magic) Methods

Method	Purpose
`__init__`	Constructor, runs on object creation
`__str__`	String shown by print() / str()
`__repr__`	Unambiguous representation for debugging
`__len__`	Enables len(obj)
`__eq__`	Enables obj1 == obj2

6. Object-Oriented Programming — Deeper Concepts

Encapsulation

```
class BankAccount:
    def __init__(self, balance):
        self._balance = balance     # convention: "protected"
        self.__pin = 1234           # name-mangled: "private"

    @property
    def balance(self):
        return self._balance
```

Python has no true private members — a single underscore signals "internal use" by convention, while a double underscore triggers name mangling to discourage accidental access.

Class Methods and Static Methods

```
class Circle:
    pi = 3.14159

    def __init__(self, radius):
        self.radius = radius

    @classmethod
    def unit_circle(cls):
        return cls(radius=1)

    @staticmethod
    def is_valid_radius(r):
        return r > 0
```

Abstract Base Classes

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        ...

class Square(Shape):
    def __init__(self, side):
        self.side = side
    def area(self):
        return self.side ** 2
```

7. Error Handling & File I/O

try / except / finally

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"Error: {e}")
except (TypeError, ValueError):
    print("Bad input type")
else:
    print("Ran fine, no exception")
finally:
    print("Always runs (cleanup)")
```

Raising Custom Exceptions

```
class InsufficientFundsError(Exception):
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError("Not enough funds")
    return balance - amount
```

Reading and Writing Files

```
# Always use "with" — it auto-closes the file
with open("data.txt", "r") as f:
    contents = f.read()

with open("out.txt", "w") as f:
    f.write("Hello, file!\n")

import json
with open("data.json") as f:
    data = json.load(f)
```

8. Iterators, Generators & Decorators

Generators

```
def count_up_to(n):
    i = 1
    while i <= n:
        yield i        # pauses and returns a value
        i += 1

for num in count_up_to(5):
    print(num)
```

Generators produce values lazily, one at a time, instead of building the whole sequence in memory — useful for large or infinite sequences.

Generator Expressions

```
squares = (x**2 for x in range(1000000))   # lazy, memory-efficient
total = sum(x**2 for x in range(10))
```

Decorators

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.time()-start:.4f}s")
        return result
    return wrapper

@timer
def slow_add(a, b):
    time.sleep(1)
    return a + b
```

A decorator wraps a function to add behavior (logging, timing, caching, access control) without modifying the function's own code.

9. Best Practices & Quick Reference

Style & Conventions (PEP 8)

- Use `snake_case` for functions/variables, `PascalCase` for classes.
- 4 spaces per indent level, never tabs.
- Keep lines under ~79-99 characters.
- Write docstrings for public functions, classes, and modules.
- Prefer explicit code over clever one-liners — readability counts.

Common Pitfalls

- Mutable default arguments (`def f(x=[])`) persist across calls.
- Modifying a list while iterating over it skips elements — iterate over a copy instead.
- Comparing floats with == — use `math.isclose()` instead.
- Catching bare `except:` hides real bugs — catch specific exceptions.

Virtual Environments & Packages

```
python -m venv venv
source venv/bin/activate      # Windows: venv\Scripts\activate
pip install requests
pip freeze > requirements.txt
pip install -r requirements.txt
```

Handy Built-ins Cheat Sheet

Function	What it does
`enumerate(iterable)`	Pairs items with an index: (0, item)
`zip(a, b)`	Pairs elements from two iterables together
`map(func, iterable)`	Applies func to every item, returns iterator
`filter(func, iterable)`	Keeps items where func(item) is True
`sorted(iterable, key=...)`	Returns a new sorted list
`any() / all()`	True if any/all elements are truthy

End of notes — happy coding!